LAB211 PROJECT REPORT | LAB 1

Mountain Hiking Challenge Registration

J1.L.P0027 | UML, Algorithms, and Test Evidence

Document field	Value
Student	Nguyen Dinh Chuong (Nguyễn Đình Chương)
Student ID	SE204330
Course	LAB211 - OOP with Java Lab
Release	Portfolio review v1.0 - 21 July 2026
Repository	https://github.com/KiritoMainBro88/SU26-LAB211

Scope intentionally limited to the lecturer's review priorities: final UML, implemented algorithms, test cases, and reproducible evidence.

1. Executive Summary

A layered Java console application for FPT University student mountain-hiking registration. It validates identity and contact data, calculates sponsored tuition, supports nine required workflows, aggregates registrations by mountain, and persists records in the object format requested by the assignment.

1.1 Verification snapshot

Check	Result	Evidence
Java compatibility compile	PASS	javac targeting Java 8, UTF-8, and -Xlint:all
Executable deep verification	34 passed, 0 failed	labs/lab1/testcases/Lab1DeepVerification.java
Import audit	PASS	0 wildcard, 0 unresolved, 0 definitely unused explicit imports
Report/Audit metadata	ALIGNED	J1.L.P0027, SE204330, 21 July 2026
Required functions traced	9/9	Section 5 traceability matrix

1.2 Evidence interpretation

The runnable deep harness was recompiled and executed during this review: 34 checks passed and none failed. The smaller verification_results.txt file is retained as earlier regression evidence (13 passed), but is not counted in the current total.

2. UML Design

This diagram is regenerated from the final source structure and names only implemented classes, interfaces, and relationships relevant to grading and maintenance.

J1.L.P0027 - Final UML Class Relationships

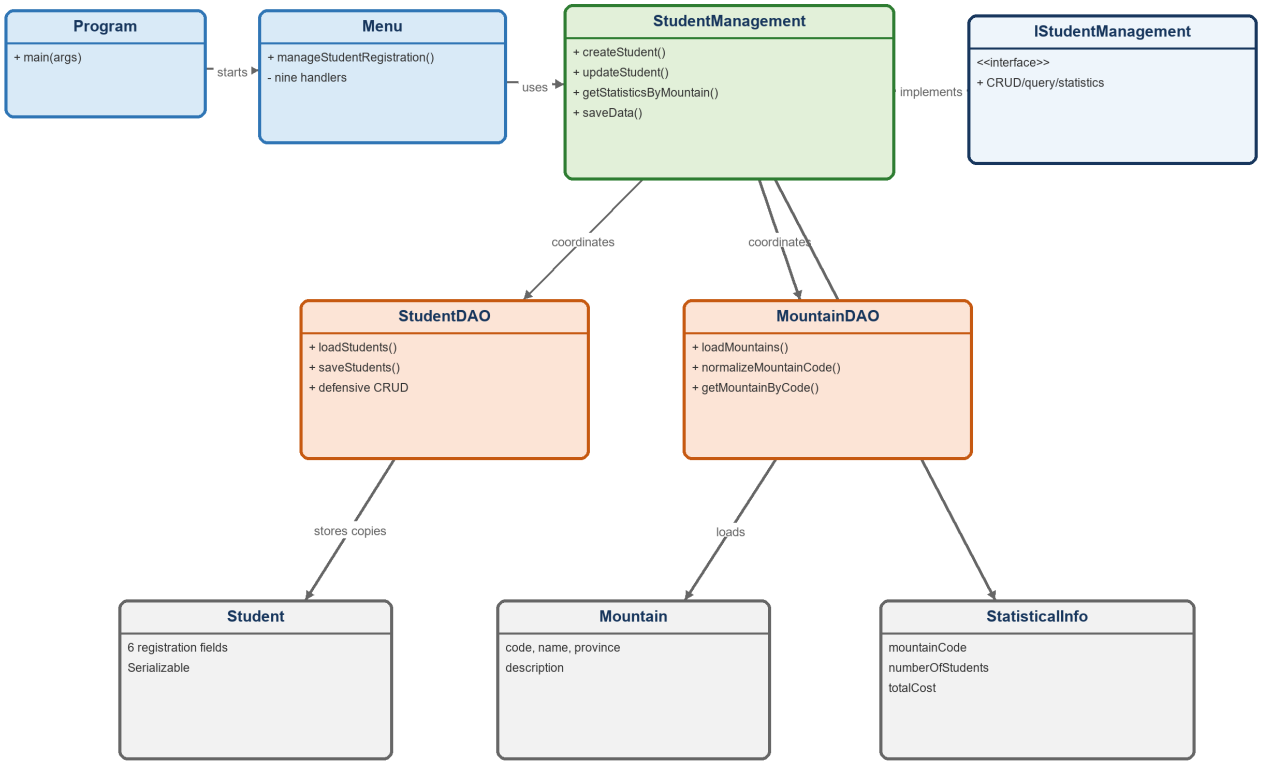

Figure 1. Final class relationships for J1.L.P0027.

2.1 Responsibility matrix

Class / Interface	Final responsibility
Program	Application entry point; loads data and starts the menu loop.
Menu	Routes the nine functions and owns console presentation only.
IStudentManagement	Focused business contract for CRUD, queries, statistics, and persistence.
StudentManagement	Coordinates validation, fee calculation, DAOs, and user-facing outcomes.
StudentDAO	Owns registration state, defensive copies, saved-state tracking, and object persistence.
MountainDAO	Loads MountainList.csv and resolves normalized mountain codes.
Student / Mountain / StatisticalInfo	Separate entities for registrations, reference data, and aggregation results.

2.2 OOP evidence

- Encapsulation: StudentDAO protects mutable Student records through deep defensive copies.
- Abstraction: IStudentManagement exposes use cases without persistence details.
- Responsibility separation: presentation, business, data access, utilities, and entities remain distinct.

3. Implemented Algorithms

Pseudocode reflects the final implementation rather than an idealized alternative. Complexity statements account for the list-based structures used by these assignment-sized applications.

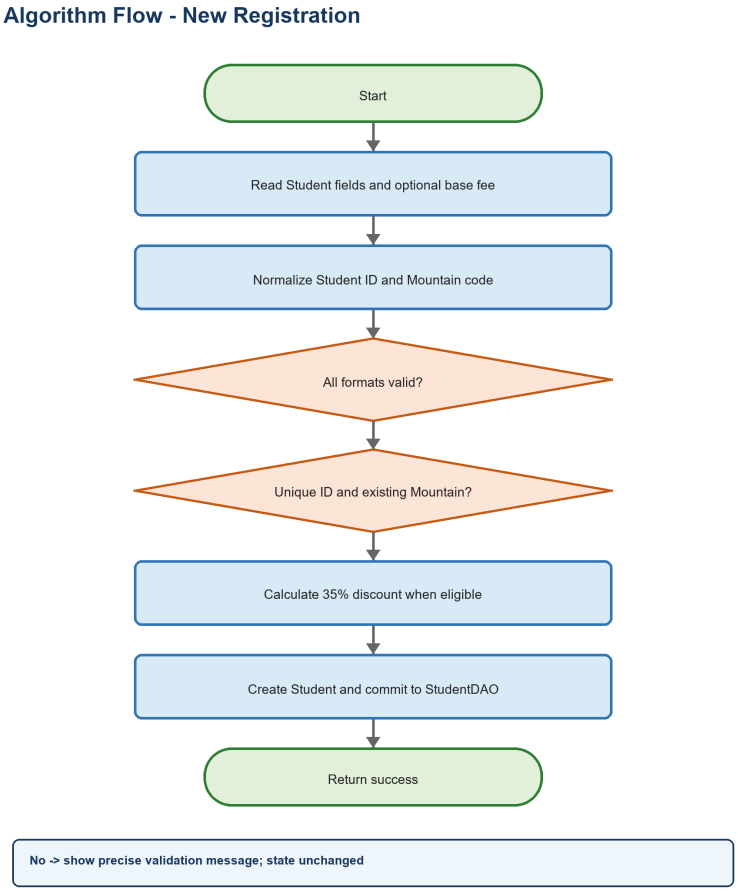

Figure 2. Representative decision flow from the final implementation.

A1 - Create a registration

Purpose: Reject invalid state before a Student reaches persistent in-memory data.

```
01  normalize studentId and mountainCode
02  if studentId/name/phone/email is invalid: REJECT
03  if mountainCode does not exist or studentId is duplicated: REJECT
04  if baseFee is not finite or baseFee <= 0: REJECT
05  fee = baseFee * 0.65 when phone is Viettel/VNPT; otherwise baseFee
06  create Student and add a defensive copy to StudentDAO
07  mark data as changed and return success
```

Complexity: O(n + m) time for duplicate and mountain lookup; O(1) additional space.

A2 - Safe partial update and tuition recalculation

Purpose: Preserve blank fields and the original custom base-fee basis.

```
01  find current Student by immutable ID; reject when missing
02  for each editable field: blank -> old value; otherwise -> candidate value
03  validate the complete candidate before changing state
04  if a new base fee exists: calculate from that fee
05  else if phone changed: recover old base fee, then recalculate discount
06  replace the record only after every check succeeds
```

Complexity: O(n + m) time; O(1) additional space excluding the candidate object.

A3 - Statistics by mountain

Purpose: Show only mountains that have registrations, with count and total tuition.

```
01  create insertion-ordered map: mountainCode -> StatisticalInfo
02  for each Student:
03      create a zeroed bucket when the code is first seen
04      increment participant count and add tuition fee
05  return the map values in deterministic first-seen order
```

Complexity: O(n) expected time and O(k) space, where k is the number of used mountains.

A4 - Defensive object-file load/save

Purpose: Prevent malformed serialization or failed replacement from destroying valid state.

```
01  LOAD: deserialize into temporary state and validate every Student and unique ID
02  LOAD: replace active memory only after complete success
03  SAVE: validate the complete current list before opening output
04  SAVE: write a copied list to a temporary sibling file
05  SAVE: atomically replace registrations.dat when supported
```

Complexity: O(n) time and O(n) temporary memory.

4. Test Cases and Executed Evidence

4.1 Execution record

Artifact	Status	Interpretation
Source compilation	PASS	All production and deep-test Java files compile for Java 8.
Deep harness	34/34 PASS	Re-executed 21 July 2026; exit code 0.
Earlier regression record	13/13 PASS	Historical result; not added to the current 153-check claim.
State preservation	PASS	Failure paths assert memory/file content, not only return values.

4.2 Representative test matrix

ID	Scenario	Input / action	Expected result	Actual
L1-T01	Valid custom-fee registration	SE123456, valid contact, MT01, 10,000,000	Added; non-discount phone keeps 10,000,000	PASS
L1-T02	Discount after phone-only update	Change phone to 0961234567; base fee blank	Preserve 10,000,000 basis; fee becomes 6,500,000	PASS
L1-T03	Student ID partitions	AB123456, SE12345A, duplicate se123456	All rejected without changing valid state	PASS
L1-T04	Name boundaries	2, 20, and 21 characters	2/20 accepted; 21 rejected	PASS
L1-T05	Phone and numeric safety	Unknown prefix, NaN, Infinity, zero	All rejected	PASS
L1-T06	Mountain normalization	1, 01, MT01, mt01	All resolve to MT01	PASS
L1-T07	Defensive query result	Mutate Student returned by getAllStudents	Manager state remains unchanged	PASS
L1-T08	Malformed serialized list	Valid Student plus non-Student item	Load fails and preserves memory	PASS

4.3 Coverage strategy

- Equivalence partitions: valid, invalid format, duplicate, missing reference, and empty/null inputs.
- Numeric boundaries: lower/upper limits plus NaN and positive infinity for double values.
- State assertions: rejected operations preserve the prior object list or target file.
- Persistence round-trips: valid object/text data reloads with essential values preserved.
- Encapsulation checks: mutations to returned objects/lists cannot alter managed state.

5. Requirement-to-Source Traceability

Every visible assignment function maps to a final handler, implementation method, and executable or directly inspectable verification artifact.

Requirement	Menu / handler	Final implementation	Evidence
1. New Registration	Menu.newRegistration	StudentManagement.createStudent	L1-T01, T03-T06
2. Update Registration	Menu.updateRegistration	StudentManagement.updateStudent	L1-T02
3. Display Registered List	Menu.displayRegisteredList	StudentManagement.getAllStudents	Defensive-copy checks
4. Delete Registration	Menu.deleteRegistration	StudentManagement.deleteStudent	Null/not-found checks
5. Search by Name	Menu.searchParticipantsByNam e	StudentManagement.searchByName	Case-insensitive search
6. Filter by Campus	Menu.filterDataByCampus	StudentManagement.filterByCampus	Valid/invalid campus
7. Statistics by Location	Menu.displayStatisticsByMounta in	StudentManagement.getStatisticsByMounta in	Aggregation checks
8. Save Data	Menu.saveData	StudentDAO.saveStudents	Save/reload and failure preservation
9. Exit	Menu.exitProgram	StudentManagement.isSaved/saveData	Save-failure cancellation trace

5.1 Import and source hygiene

Audit item	Result	Basis
Incorrect/unresolved imports	0	Complete production + test compilation succeeded.
Wildcard imports	0	Repository-wide source scan.
Definitely unused explicit imports	0	Imported simple names are referenced in their units.
Generated binaries committed	0	.gitignore excludes build, dist, class, jar, private IDE state, and logs.

5.2 Known limitations and deliberate trade-offs

- Java object serialization is intentionally assignment-specific and is not a long-term interchange format.
- Mountain reference parsing is designed for the supplied CSV shape; it is not a general RFC 4180 library.
- Mutable Mountain objects are exposed only to the read-only presentation path; Student records use defensive copies.

5.3 Reproduction

From the repository root on Windows:

```
PowerShell -ExecutionPolicy Bypass -File .\scripts\verify-all.ps1
```

GitHub Actions invokes the same script on every push and pull request. The workflow pins actions to reviewed commit SHAs and grants read-only repository permission.

Conclusion

The final J1.L.P0027 source implements all 9 required functions, compiles for Java 8, and passes 34 executable deep checks. The UML, algorithms, test evidence, and AI Audit Log use the same identifiers, counts, paths, and verification date.